



**Computational
Data Science LAB**

TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation

윤수아 / 2026.05.19

관련문서	S. G. Müller and F. Hutter, “TrivialAugment: Tuning-free yet state-of-the-art data augmentation,” in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 774–782.
논의사항 및 의문사항	
기타사항	



CONTENTS

1. Introduction
 2. Related Work
 3. Methods
 4. Experiments
 5. Conclusions
- 
- 

01 | Introduction

- 기존 자동 증강 기법의 한계
 - ✓ 높은 복잡성 및 비용 : 최적의 증강 정책 탐색에 막대한 계산 자원과 시간이 소요
 - ✓ 튜닝 필요성 : 각 데이터셋과 모델에 맞춰 복잡한 정책이나 하이퍼파라미터 튜닝 필요
 - ✓ 모델의 복잡성과 성능 trade-off: 성능을 높이기 위해 복잡성을 감수 필요
- 기존 한계점을 해결하기 위해 **각 작업에 대해 별도의 증강 정책을 학습하지 않아도 훨씬 비용이 큰 증강 방법들과 비슷한 성능을 보이는 매우 단순한 증강 방법을 제안**

02 | Related Work

- AutoAugment (AA)
 - ✓ 고비용으로 강화 학습을 사용해 이미지에 적용할 최적의 증강 정책을 탐색하는 방법
- Population-based Augmentation (PBA)
 - ✓ 여러 워커가 각자 다른 증강 정책을 사용하며 학습 중에 진화적으로 업데이트하는 방식으로 정책을 탐색하는 방법
- Online Hyper-Parameter Learning for Auto-Augmentation (OHL)
 - ✓ 여러 워커를 사용하며, 부분적인 학습 후 정확도를 보상으로 받아 증강 정책의 파라미터를 강화 학습으로 튜닝하는 방법
- Adversarial AutoAugment (Adv. AA)
 - ✓ 하나의 배치(batch)를 여러 워커에 복사하여 각기 다른 증강을 적용하고, 학습 정확도를 낮추는 정책에 보상을 주어 점진적으로 강한 증강을 탐색하는 방법

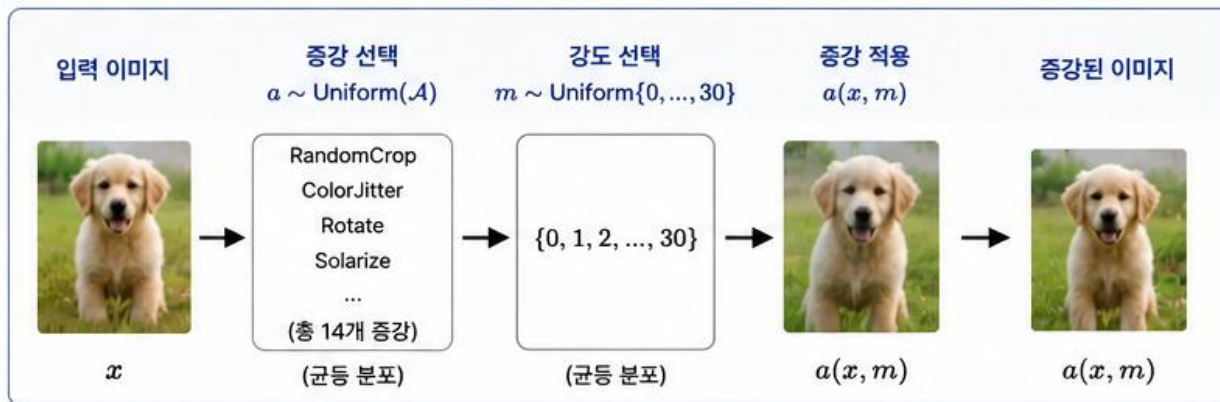
02 | Related Work

- RandAugment (RA)
 - ✓ 증강의 종류와 강도를 조절하는 단 두 개의 파라미터만 튜닝하여, 복잡한 탐색 공간을 크게 줄인 실용적인 방법
- Fast AutoAugment (Fast AA)
 - ✓ 학습 초기에 원본 데이터로 추론 증강 정책을 찾아내고, 이를 결합하여 학습 증강 정책을 만드는 방식으로 탐색 비용을 줄인 방법
- UniformAugment (UA)
 - ✓ 증강 횟수를 고정하고 각 증강의 강도를 균등하게 샘플링하는 탐색-프리(search-free) 방식의 증강 기법

03 | Methods

- TrivialAugment(TA)

- ✓ 입력으로 이미지 x 와 증강집합 A 만을 입력으로 받아 파라미터 탐색 없이 단순히 증강시키는 방법
- ✓ A 에서는 증강을 균일하게 무작위로 샘플링하고, $\{0, \dots, 30\}$ 범위에서 균일하게 무작위로 샘플링된 강도 m 으로 주어진 이미지 x 에 이 증강을 적용
 - a : 이미지를 증강된 이미지로 매핑하는 함수
 - m : 이상 강도 파라미터
- ✓ TA는 각 이미지에 하나의 증강만 적용하기 때문에 최종 데이터 분포는 여러 증강 방식으로 생성된 데이터 분포들의 평균으로 볼 수 있음



Algorithm 1 TrivialAugment Procedure

```
1: procedure TA( $x$ : image)
2:   Sample an augmentation  $a$  from  $\mathcal{A}$ 
3:   Sample a strength  $m$  from  $\{0, \dots, 30\}$ 
4:   Return  $a(x, m)$ 
5: end procedure
```

04 | Experiments

- 실험 방법

- ✓ 기존 SOTA 연구들과 TA의 성능 비교 실험 진행

- 1) 기존 방법론들의 논문에서 언급된 모델과 세부 설정을 기반으로 비교
- 2) 기존 방법론들의 모델 설정을 재현한 실제 결과 값을 기반으로 비교
- 3) 기존 방법론들과 GPU 계산 비용 비교

- ✓ 증강 기법들의 증강 공간 영향력에 대한 실험 진행

- 1) 기존 연구들의 증강 공간을 활용했을 때의 TA 성능 비교
- 2) 랜덤 증강 공간 설정에 따른 TA 성능 비교
- 3) 랜덤 증강 강도 설정에 따른 TA 성능 비교

- 기본 설정

- ✓ Accuracy를 평가지표로 사용

04 | Experiments

Comparison to State-of-the-Art

- Comparison to Published Results

✓ 이전 연구들과 동일한 데이터셋과 모델을 활용해 비교 실험 진행

	Default	PBA	Fast AA	AA	RA	UA	TA (Wide)
CIFAR-10							
Wide-ResNet-40-2	96.16 ± .08	-	96.4	96.3	-	96.25	96.32 ± .05
Wide-ResNet-28-10	97.03 ± .07	97.4	97.3	97.4	97.3	97.33	97.46 ± .06
ShakeShake-26-2x96d	97.54 ± .07	98.0	98.0	98.0	98.0	98.10	98.21 ± .06
PyramidNet	97.95 ± .05	98.5	98.5	98.3	98.5	98.5	98.58 ± .04
CIFAR-100							
Wide-ResNet-40-2	78.42 ± .31	-	79.4	79.3	-	79.01	79.86 ± .19
Wide-ResNet-28-10	82.22 ± .25	83.3	82.7	82.9	83.3	82.82	84.33 ± .17
ShakeShake-26-2x96d	83.28 ± .14	84.7	85.4	85.7	-	85.00	86.19 ± .15
SVHN Core							
Wide-ResNet-28-10	97.12 ± .05	-	-	98.0	98.3	-	98.11 ± .03
SVHN							
Wide-ResNet-28-10	98.67 ± .02	98.9	98.8	98.9	99.0	-	98.9 ± .02
ImageNet							
ResNet-50	77.20 ± .32 (93.43 ± .11)	-	77.6 (93.7)	77.6 (93.8)	77.6 (93.8)	77.63 (-)	78.07 ± .27 (93.92 ± .09)

	Adv. AA	TA (Wide)
CIFAR-10		
Wide-ResNet-28-10	98.10 ± .15	98.04 ± .06
ShakeShake-26-2x96d	98.15 ± .12	98.12 ± .12
CIFAR-100		
Wide-ResNet-28-10	84.51 ± .18	84.62 ± .14
ShakeShake-26-2x96d	85.90 ± .15	86.02 ± .13

04 | Experiments

Comparison to State-of-the-Art

- Comparison of Reproduced Results in a Fixed Training Setup
 - ✓ CIFAR 데이터셋을 활용하여 Wide-ResNet-28-10(a), Wide-ResNet-40-2(b) 모델을 사용하여 기존 연구를 재현
 - ✓ SVHN Core(c)에서는 Wide-ResNet-28-10 모델을 사용하여 기존 연구를 재현
 - ✓ 각자 논문에 보고된 결과 대비 상대적인 성능 차이를 표기

(a)

WRN-28-10	CIFAR-10	CIFAR-100
AA	97.31 $\pm$.22 (-.09)	82.91 $\pm$.41 (+.01)
FAA	97.43 $\pm$.09 (+.13)	83.27 $\pm$.13 (+.57)
RA	97.12 $\pm$.14 (-.18)	83.1 $\pm$.32 (-.20)
UA (UA)	97.46 $\pm$.14 (+.13)	83.08 $\pm$.27 (+.26)
UA (RA)	97.44 $\pm$.09	83.36 $\pm$.18
TA (RA)	97.46 $\pm$.09	83.54 $\pm$.12
TA (Wide)	97.46 $\pm$.06	84.33 $\pm$.17

(b)

WRN-40-2	CIFAR-10	CIFAR-100
AA	96.38 $\pm$.10 (+.08)	79.66 $\pm$.17 (+.36)
FAA	96.39 $\pm$.06 (-.01)	79.79 $\pm$.21 (+.39)
UA (UA)	96.42 $\pm$.04 (+.17)	79.74 $\pm$.15 (+.73)
UA (RA)	96.45 $\pm$.06	79.95 $\pm$.20
TA (RA)	96.62 $\pm$.09	79.99 $\pm$.16
TA (Wide)	96.32 $\pm$.05	79.86 $\pm$.19

(c)

WRN-28-10	SVHN Core
AA	97.99 $\pm$.06 (-.01)
RA	98.06 $\pm$.04 (-.24)
TA (RA)	98.05 $\pm$.02
TA (Wide)	98.11 $\pm$.03

04 | Experiments

Comparison to State-of-the-Art

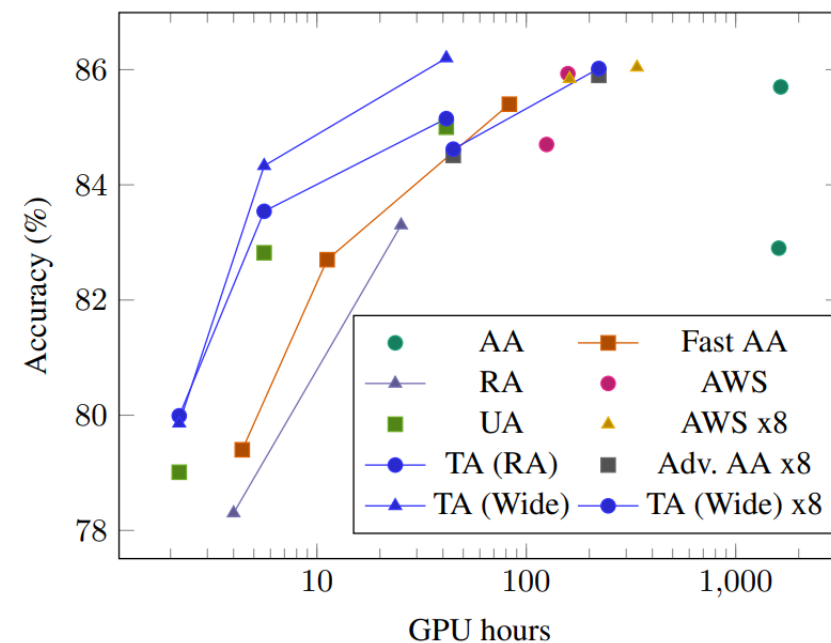
- Comparison of Reproduced Results in a Fixed Training Setup

- ✓ RA에 비해 TA가 더 적은 계산 비용으로 더 좋은 결과를 보임
- ✓ RA는 91번의 학습을 통해 도출된 결과이지만, TA는 단 한번의 학습 과정을 통해 도출한 결과임
 - Brute-Force : 가능한 N, M조합을 모두 시도한 RA

Method	Acc.
Brute-Force RA	$96.42 \pm .09$
TA (RA)	$96.62 \pm .09$

- Comparison by Total Compute Costs

- ✓ RTX2080 Ti GPU를 활용해 서로 다른 증강 방법의 컴퓨팅 비용 비교
- ✓ Wide-ResNet 계열 모델들과 Shake-Shake-26-2x96d 모델을 활용
 - TA (RA) : 증강 기법과 강도 범위를 RA에서 사용한 것들로 적용한 TA
 - TA (Wide) : 본논문에서 제시하는 Wide범위의 증강 기법을 적용한 TA
 - TA (Wide) x8 : 배치 증강 기법을 적용한 TA



04 | Experiments

Understanding the Minimal Requirements of TA

- TA with Different Hand-Picked Augmentation Spaces
 - ✓ 기존 연구에서 제안된 augmentation space에 따라 TA가 어떻게 동작하는지 분석
 - ✓ 기존 여러 증강 방법론들의 space를 고려하여 수작업으로 새로운 7가지 augmentation space를 정의
 - ✓ Wide-ResNet-28-10 모델을 사용한 SVHN Core와 CIFAR-10 실험에서 TA가 Full augmentation space에서 다른 augmentation space들보다 성능이 낮게 나타남

Augmentation space	SVHN Core	CIFAR-10
Full	97.63 $\pm$.06	97.24 $\pm$.03
AA	98.04 $\pm$.02	97.47 $\pm$.11
AA - {Invert}	97.97 $\pm$.08	97.55 $\pm$.06
RA	98.05 $\pm$.02	97.46 $\pm$.09
Wide	98.11 $\pm$.03	97.46 $\pm$.06
UA	98.06 $\pm$.04	97.42 $\pm$.07
OHL	98.10 $\pm$.02	97.45 $\pm$.05

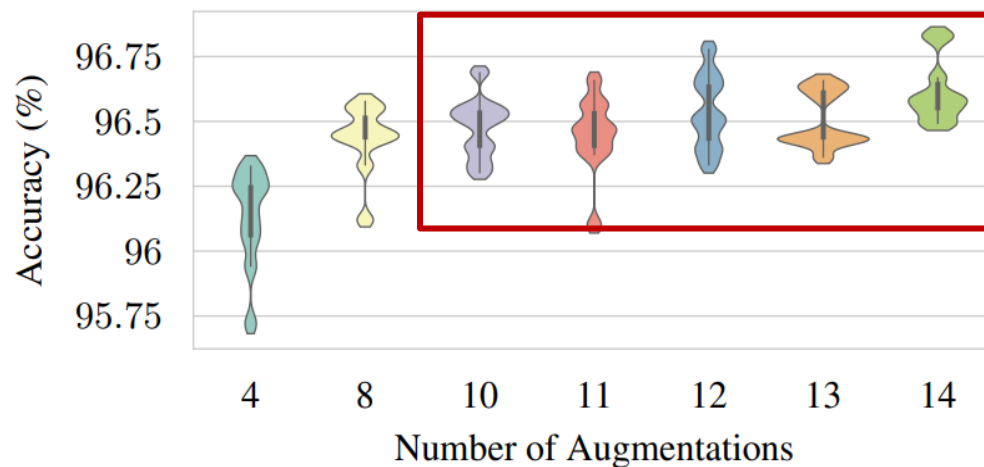
Table 6: Evaluation of TA on SVHN Core and CIFAR-10 with a set of 7 different augmentation spaces. Note that RA = AA - {SamplePairing, Invert, Cutout} and UA = AA - {SamplePairing}.

PIL operation	range	PIL operation	range
identity	-	auto_contrast	-
equalize	-	rotate	-30° - +30° (-135° - +135°)
solarize	0 - 256 (0 - 256)	color	0.1 - 1.9. (0.01 - 2.)
posterize	4 - 8 (2 - 8)	contrast	0.1 - 1.9. (0.01 - 2.)
brightness	0.1 - 1.9. (0.01 - 2.)	sharpness	0.1 - 1.9. (0.01 - 2.)
shear_x	0.0 - 0.3 0.0 - 0.99	shear_y	0.0 - 0.3 0.0 - 0.99
translate_x	0 - 10 (0 - 32)	translate_y	0 - 10 (0 - 32)
cutout	0 - 0.2	invert	-
flip_lr	-	flip_ud	-
sample_pairing	0.0 - 0.4	blur	-
smooth	-		

04 | Experiments

Understanding the Minimal Requirements of TA

- TA's Behavior With Randomly Pruned Augmentation Spaces
 - ✓ 수작업이 아닌 RA augmentation space의 14개 augmentation 중 일부를 무작위로 선택했을 때의 성능을 비교
 - ✓ CIFAR-10 데이터셋과 Wide-ResNet-40-2 모델을 사용하여 augmentation subset 크기에 따른 성능과 분산을 분석
 - 각 subset 크기마다 10번의 평가를 수행하였고, 매 평가마다 augmentation들을 무작위로 선택
 - ✓ 증강 기법 개수가 줄어들수록 성능은 감소하지만, 감소 폭이 완만하여 전체 14개 중 4개정도는 제거해도 원래 성능과 유사
 - ✓ 증강 기법 개수가 줄어들수록 성능 분산이 증가하는 경향 존재
 - subset 크기가 작아질수록 어떤 augmentation이 선택되는지에 따른 랜덤성 영향이 커짐을 의미



04 | Experiments

Understanding the Minimal Requirements of TA

- The Impact of the Set of Strengths on TA's Performance
 - ✓ TA가 샘플링하는 augmentation strength(강도) 집합의 영향을 분석
 - ✓ RA augmentation space 상에서 Wide-ResNet-28-10 모델과 함께 원래 가능한 strength 집합 $\{0, \dots, 30\}$ 의 다양한 subset을 사용한 TA 성능을 분석
 - ✓ CIFAR-100에서는 $\{30\}$ 만 사용하는 경우 성능이 크게 감소했으며, SVHN Core에서는 $\{30\}$ 만 사용할 때 성능이 더 좋아짐
 - ✓ 모든 31개의 strength를 사용하는 것과 비교했을 때 strength를 3개 혹은 2개로 줄여도 성능 차이가 크지 않은 결과를 보임
 - 강한 augmentation과 약한 augmentation이 혼합되어 있는 것이 중요함을 시사

Strengths	CIFAR-10	CIFAR-100	SVHN Core
$\{30\}$	97.45 $\pm$.05	82.98 $\pm$.22	98.16 $\pm$.03
$\{0, 30\}$	97.51 $\pm$.08	83.46 $\pm$.10	98.02 $\pm$.02
$\{0, 15, 30\}$	97.46 $\pm$.06	83.43 $\pm$.24	98.04 $\pm$.03
$\{0, \dots, 30\}$	97.46 $\pm$.09	83.54 $\pm$.12	98.05 $\pm$.02

05 | Conclusion

- 본 논문을 한마디로 정의하면 무엇인가?
 - ✓ 파라미터 탐색 없이 간단한 랜덤 선택 방법으로 좋은 성능을 달성한 증강 기법
- 본 논문에서 놀랍고, 새로운 점은 무엇인가?
 - ✓ 파라미터 탐색이 없는 모델이 더 성능이 좋았던 점
 - ✓ Augmentation space의 정의가 중요하다는 점
- 본인이 예상한 바와 어떤 점이 다른가?
 - ✓ 기본적인 파라미터 설정도 없이 간단한 랜덤 방식으로 기존 방법들을 능가하는 성능을 달성한 점
- 본 논문과 관련된 본인의 아이디어는 무엇인가?
 - ✓ Policy기반 최적화 방법론들을 간단한 랜덤 방법론들로 대체하면 속도 측면과 성능 측면에서 훨씬 좋은 성능을 달성할 수 있을 것으로 기대됨

Q&A

감사합니다.

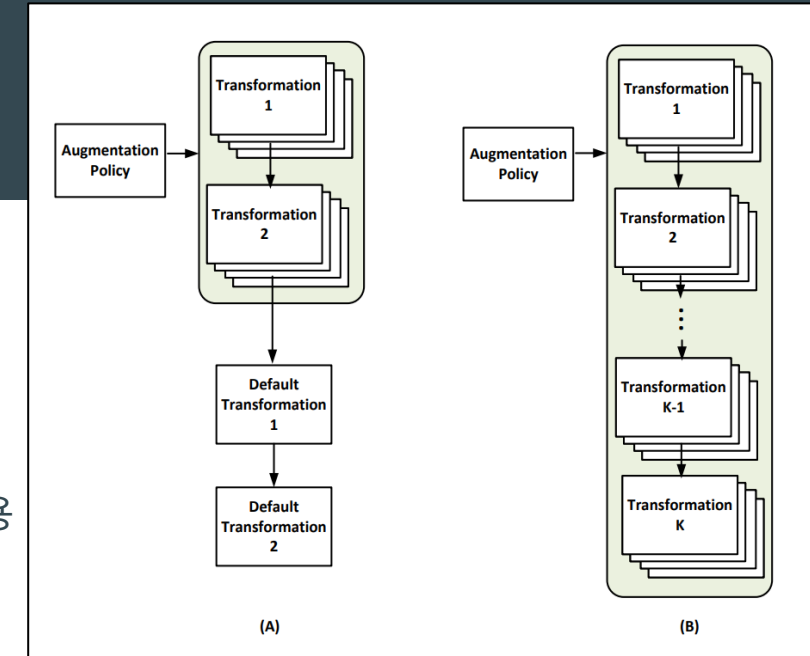
06 | Deep AutoAugment

- Limitations of Previous Studies

- ✓ 이전 연구들은 탐색기반 변환은 1개 또는 2개이며, 나머지 변환들은 수작업 변환이 필요한 한계점이 존재
 - 탐색한 증강 기법에 Crop, Flip, Cutout과 같은 기본적인 방법을 추가로 증강시켜 사용

- Key idea

- ✓ 데이터 증강은 원본 데이터의 의미와 분포를 유지하면서 다양한 형태의 데이터를 생성해 부족한 데이터를 보완하는 과정
- ✓ 하나의 데이터를 여러 번 증강하여 특정 변환 유형 하에서 생성된 데이터 분포가 전체 데이터셋의 분포와 유사해야 함
- ✓ 증강 데이터의 분포가 전체 데이터셋 분포와 가까워질수록 증강 데이터의 gradient도 원본 데이터 배치의 gradient 방향에 가까워져야 함
- DeepAA에서는 데이터 증강 정책 탐색을 regularized gradient matching 문제로 정의



06 | Deep AutoAugment

- Regularized gradient matching
 - ✓ 하나의 이미지를 여러 번 증강하고, 증강된 이미지의 gradient가 원본 데이터의 gradient 방향으로 향하도록 만드는 방식
 - ① 하나의 학습 데이터를 augmentation policy에 따라 여러 번 증강하여 augmented training batch를 생성
 - ② Validation set에서 원본 데이터 배치를 샘플링하여 validation batch를 생성
 - ③ augmentation을 통해 augmented training batch의 gradient가 validation batch의 gradient 방향으로 가까워지도록 유도
 - ④ validation 데이터 gradient와 증강 데이터 gradient 사이의 cosine similarity를 최대화하는 augmentation policy를 탐색
 - 즉, 효과적인 augmentation은 데이터 분포를 보존해야 하며, 증강 이미지 분포가 validation set 분포와 alignment되어 학습 gradient 방향이 validation gradient 방향과 가까워져야 한다는 것

06 | Deep AutoAugment

- Limitations of Previous Studies

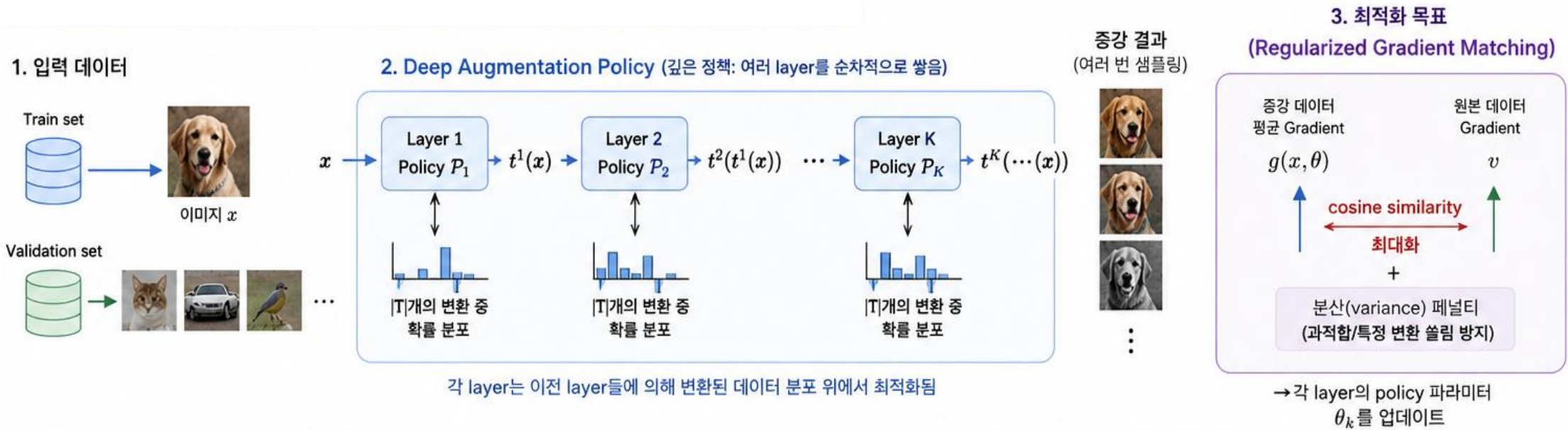
- ✓ Augmentation layer가 깊어질수록 ($K \geq 5$) search space가 매우 커지는 문제 존재
 - 좋은 policy를 탐색으로 찾기 어려워짐 → 고차원 search space에서 좋은 policy가 희소해짐
 - policy parameter 차원이 K 에 비례해 증가 → 최적화 계산량 증가

- Key idea

- ✓ DeepAA는 Augmentation layer를 stack progressively하게 쌓는 방식을 제안
 - 각 augmentation layer는 이전 layer들에 의해 변환된 데이터 분포 위에서 최적화
 - 거대한 search space에서 직접 sub-policy를 샘플링할 필요가 없어지고, policy parameter 수 역시 $|T|^K$ 에서 $|T|$ 수준으로 감소

06 | Deep AutoAugment

- Architecture



06 | Deep AutoAugment

- Search Space

- ✓ 한 개의 augmentation policy $\mathcal{P}$ 는 여러 개의 sub-policy로 구성
- ✓ 최종 augmentation policy는 $\mathcal{P} = \{\mathcal{P}_1, \dots, \mathcal{P}_K\}$ 형태의 K개의 sequential augmentation layer로 구성

$$\mathcal{P}_k := p_{\theta_k}(n|\mathcal{P}_1, \dots, \mathcal{P}_{k-1})$$

- $n : \mathbb{T}$ 에서의 변환 인덱스를 의미
 - 가능한 transformation 개수 = $|\mathbb{M}| \cdot |\mathbb{O}| = \text{증강 강도 범위 수} \cdot \text{증강 기법 수}$
- ✓ $p_{\theta_k}(n|\mathcal{P}_1, \dots, \mathcal{P}_{k-1})$ 를 p_{θ_k} 로 간소화하여 표현

06 | Deep AutoAugment

- Augmentation policy search via regularized gradient matching
 - ✓ 하나의 데이터 포인트 x 가 정책 p_θ 를 따라 여러 번 증강된다고 가정
 - ✓ 아래 식을 활용해 증강 데이터의 gradient와 원본 데이터 배치의 gradient 사이의 cosine similarity를 최대화하여 augmentation policy를 최적화

$$\begin{aligned}\theta &= \arg \max_{\theta} \text{cosineSimilarity}(v, g(x, \theta)) \\ &= \arg \max_{\theta} \frac{v^T \cdot g(x, \theta)}{\|v\| \cdot \|g(x, \theta)\|}\end{aligned}$$

- $\|\cdot\|$: L2-norm
- $g(x, \theta)$: 증강으로부터 생성된 평균 gradient, 데이터 x 와 정책 파라미터 θ 의 함수
- v : 원본 데이터 배치의 gradient를 의미

- ✓ Policy parameter는 gradient ascent방법을 통해 업데이트

$$\theta \leftarrow \theta + \eta \nabla_{\theta} \text{cosineSimilarity}(v, g(x, \theta))$$

06 | Deep AutoAugment

- Policy search for one layer

- ✓ $\mathcal{P} = \{p_\theta\}$ 로 정의된 p_θ 를 따라 하나의 데이터 포인트 x 에 augmentation을 적용하면, 생성되는 평균 gradient는 가능한 모든 transformation $\mathbb{T}$ 에 대해 각각의 확률 $p(\theta)$ 를 고려하여 평균을 내는 방식으로 계산

$$\begin{aligned} g(x; \theta) &= \sum_{n=1}^{|\mathbb{T}|} p_\theta(n) \nabla_w L(t_n(x); w) \\ &= G(x) \cdot p_\theta \end{aligned}$$

- $G(x) = [\nabla_w L(t_1(x); w), \dots, \nabla_w L(t_{|\mathbb{T}|}(x); w)]$ is a $D \times |\mathbb{T}|$ Jacobian matrix
- $p_\theta = [p_\theta(1), \dots, p_\theta(|\mathbb{T}|)]^T : |\mathbb{T}|$ dimensional categorical distribution
- $L(x; w) : \text{data point } x \text{의 classification loss}$
- $w \in \mathbb{R}^D : \text{신경망의 flatten된 weight 벡터를 의미}$

$$\nabla_\theta \text{cosineSimilarity}(v, g(x; \theta)) = \nabla_\theta p_\theta \cdot r \qquad r = G(x)^T \left(\frac{v}{\|g(\theta)\|} - \frac{v^T g(\theta)}{\|g(\theta)\|^2} \cdot \frac{g(\theta)}{\|g(\theta)\|} \right)$$

- $r : \text{transformation에 대한 reward를 의미}$
- $p_\theta \cdot r : \text{정책 } p_\theta \text{하에서의 평균 reward를 의미}$

06 | Deep AutoAugment

- Policy search for multiple layers
 - ✓ $K \geq 2$ 일 경우, search space 차원이 K 에 따라 지수적으로 증가하여 증강 데이터의 평균 gradient 계산이 어려움
 - ✓ 이전 augmentation layer들에 의해 변형된 데이터 분포를 기반으로 augmentation을 점진적으로 쌓아가는 방법 사용
 - ✓ 각 policy는 $|T|$ 차원의 categorical distribution이며, k 번째 layer의 Jacobian matrix는 이전 $(k-1)$ 개의 policy layer들에 대해 평균을 취함으로써 다음과 같이 계산됨

$$G(x)^k = \sum_{n_{k-1}=1}^{|T|} \cdots \sum_{n_1=1}^{|T|} p_{\theta_{k-1}}(n_{k-1}) \cdots p_{\theta_1}(n_1) [\nabla_w L((t_1 \circ t_{n_{k-1}} \cdots \circ t_{n_1})(x); w), \cdots, \nabla_w L((t_{|T|} \circ t_{n_{k-1}} \circ \cdots \circ t_{n_1})(x); w)]$$

$$\tilde{G}^k(x) = \sum_{\tilde{n}_{k-1} \sim p_{\theta_k}} \cdots \sum_{\tilde{n}_1 \sim p_{\theta_1}} [\nabla_w L((t_1 \circ t_{\tilde{n}_{k-1}} \cdots \circ t_{\tilde{n}_1})(x); w), \cdots, \nabla_w L((t_{|T|} \circ t_{\tilde{n}_{k-1}} \circ \cdots \circ t_{\tilde{n}_1})(x); w)]$$

where $\tilde{n}_{k-1} \sim p_{\theta_{k-1}}(n), \cdots, \tilde{n}_1 \sim p_{\theta_1}(n)$.

$$\tilde{g}(x; \theta_k) = \sum_{\tilde{n}_k \sim p_{\theta_k}} \cdots \sum_{\tilde{n}_1 \sim p_{\theta_1}} \nabla_w L((t_{\tilde{n}_k} \circ \cdots \circ t_{\tilde{n}_1})(x); w).$$

Monte Carlo방법을 통해 G^k 를 근사

Monte Carlo방법을 통해 k 번째 layer의 평균 gradient를 계산

06 | Deep AutoAugment

- Policy search for multiple layers

- ✓ 앞의 k 번째 layer의 평균 gradient식에 따라 k 번째 layer의 reward는 아래와 같이 정의됨

$$\tilde{r}^k(x) = \left(\tilde{G}^k(x) \right)^T \left(\frac{v}{\|\tilde{g}_k(x; \theta_k)\|} - \frac{v^T \tilde{g}_k(x; \theta_k)}{\|\tilde{g}_k(x; \theta_k)\|^2} \cdot \frac{\tilde{g}_k(x; \theta_k)}{\|\tilde{g}_k(x; \theta_k)\|} \right).$$

- ✓ augmentation policy 과적합 방지를 위해 variance가 큰 방향으로 optimization되는 것을 피하도록 regularization을 적용

$$r^k = E_x\{\tilde{r}^k(x)\} - c \cdot \sqrt{E_x\{(\tilde{r}^k(x) - E_x\{\tilde{r}^k(x)\})^2\}}$$

→ 평균 reward에 대한 표준편차 기반 penalty를 적용

- 기대값 계산에서는 무작위로 샘플링한 16개의 이미지를 사용
- 하이퍼 파라미터 c 는 regularization 정도를 조절하며 논문에서는 1.0. 설정

- ✓ 따라서, $k \geq 2$ 인 policy P_k 의 parameter는 아래와 같이 업데이트

$$\theta \leftarrow \theta_k + \eta \nabla_{\theta_k} \text{cosineSimilarity}(v, g(\theta_k))$$

$$\nabla_{\theta} \text{cosineSimilarity}(v, g^k(x; \theta)) = \nabla_{\theta} p_{\theta_k} \cdot r^k$$